DATA STRUCTURES AND ALGORITHMS

DSA LAB RECORD

Name	S Pardhasaradhi
Register Number	11259A487
Section	S9
Subject	DSA (Data Structures and Algorithms)

DEPARTMENT OF COMPUTER SCIENCE / ENGINEERING

EXPERIMENT 1: Array Operations

AIM

To perform insertion, deletion, traversal and searching operations on a one-dimensional array.

ALGORITHM

1. Begin the program and accept the array size along with the array elements from the user.
2. Present a menu to the user with the choices: insert, delete, traverse, search or exit.
3. For insertion, read the position and the value to be inserted. Validate the position, shift all elements from that position one place to the right, place the value at the desired position and increase the count of elements.
4. For deletion, read the position of the element to be removed. Validate the position, shift every element after it one place to the left and decrease the count of elements.
5. For traversal, visit each element from the first to the last and display it on the screen.
6. For searching, compare the key value with every element of the array sequentially until a match is found or the end of the array is reached.
7. Repeat step 2 until the user chooses the exit option, then terminate the program.

PROGRAM (C)

```
#include <stdio.h>
#define MAX 100
int main(void){
    int a[MAX], n, ch, pos, value, i, key, found;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements: ");
    for (i = 0; i < n; i++) scanf("%d", &a[i]);
    do {
        printf("\n1.Insert 2.Delete 3.Traverse 4.Search 5.Exit\n");
        scanf("%d", &ch);
        switch (ch){
            case 1:
                if (n == MAX){ printf("Array is full\n"); break; }
                printf("Position and value: ");
                scanf("%d%d", &pos, &value);
                if (pos < 1 || pos > n + 1){ printf("Invalid position\n"); break; }
                for (i = n; i >= pos; i--) a[i] = a[i-1];
                a[pos-1] = value; n++;
                break;
            case 2:
                printf("Position to delete: "); scanf("%d", &pos);
                if (pos < 1 || pos > n){ printf("Invalid position\n"); break; }
                for (i = pos-1; i < n-1; i++) a[i] = a[i+1];
                n--; break;
            case 3:
                for (i = 0; i < n; i++) printf("%d ", a[i]);
                printf("\n"); break;
            case 4:
                printf("Element to search: "); scanf("%d", &key);
                found = 0;
                for (i = 0; i < n; i++)
                    if (a[i] == key){ printf("Found at position %d\n", i+1); found = 1; break; }
                if (!found) printf("Not found\n");
                break;
        }
    } while (ch != 5);
    return 0;
}
```

OUTPUT

```
Enter number of elements: Enter elements:
1.Insert 2.Delete 3.Traverse 4.Search 5.Exit
Position and value:
1.Insert 2.Delete 3.Traverse 4.Search 5.Exit
10 20 99 30 40 50

1.Insert 2.Delete 3.Traverse 4.Search 5.Exit
Element to search: Found at position 5

1.Insert 2.Delete 3.Traverse 4.Search 5.Exit
Position to delete:
1.Insert 2.Delete 3.Traverse 4.Search 5.Exit
20 99 30 40 50

1.Insert 2.Delete 3.Traverse 4.Search 5.Exit
```

RESULT

Thus, insertion, deletion, traversal and searching operations on an array were implemented and executed successfully.

EXPERIMENT 2: Structure Operations

AIM

To define a structure and read, display and process student details using an array of structures.

ALGORITHM

1. Declare a structure named Student with members to hold the roll number, name and marks.
2. Read the total number of students to be processed.
3. Run a loop to accept the roll number, name and marks of every student and store them in the array of structures.
4. Accumulate the marks of all students in order to compute the total.
5. Run another loop to display the record of each student in a tabular form.
6. Divide the accumulated total by the number of students and display the class average.
7. Terminate the program after displaying all the details.

PROGRAM (C)

```
#include <stdio.h>
struct Student {
    int roll;
    char name[30];
    float marks;
};
int main(void){
    struct Student s[50];
    int n, i;
    float total = 0;
    printf("Enter number of students: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++){
        printf("Enter roll, name and marks: ");
        scanf("%d %29s %f", &s[i].roll, s[i].name, &s[i].marks);
        total += s[i].marks;
    }
    printf("\nRoll\tName\tMarks\n");
    for (i = 0; i < n; i++)
        printf("%d\t%s\t%.2f\n", s[i].roll, s[i].name, s[i].marks);
    if (n > 0) printf("Average = %.2f\n", total / n);
    return 0;
}
```

OUTPUT

```
Enter number of students: Enter roll, name and marks: Enter roll, name and marks: Enter roll, name and marks
Roll■Name■Marks
101■Arun■85.50
102■Banu■90.00
103■Charan■78.25
Average = 84.58
```

RESULT

Thus, student records were stored, accessed and displayed successfully using structures.

EXPERIMENT 3: Stack Using Array

AIM

To implement stack operations (push, pop and display) using an array.

ALGORITHM

1. Declare an array of fixed size to represent the stack and initialize the top pointer TOP to -1, indicating an empty stack.
2. For the PUSH operation, first check whether the stack is full (overflow). If it is not full, increment TOP and store the new item at that position.
3. For the POP operation, check whether the stack is empty (underflow). If it contains elements, display the item at TOP and decrement TOP.
4. For the DISPLAY operation, check for underflow; otherwise print all elements from the TOP position down to the bottom of the stack.
5. Show the menu repeatedly and perform the chosen operation until the user selects the exit option.

PROGRAM (C)

```
#include <stdio.h>
#define MAX 50
int main(void){
    int stack[MAX], top = -1, ch, x, i;
    do {
        printf("\n1.Push 2.Pop 3.Display 4.Exit\n");
        scanf("%d", &ch);
        if (ch == 1){
            if (top == MAX-1) printf("Stack overflow\n");
            else { printf("Enter value: "); scanf("%d", &x); stack[++top] = x; }
        } else if (ch == 2){
            if (top == -1) printf("Stack underflow\n");
            else printf("Deleted: %d\n", stack[top--]);
        } else if (ch == 3){
            if (top == -1) printf("Stack is empty\n");
            else { for (i = top; i >= 0; i--) printf("%d ", stack[i]); printf("\n"); }
        }
    } while (ch != 4);
    return 0;
}
```

OUTPUT

```
1.Push 2.Pop 3.Display 4.Exit
Enter value:
1.Push 2.Pop 3.Display 4.Exit
Enter value:
1.Push 2.Pop 3.Display 4.Exit
Enter value:
1.Push 2.Pop 3.Display 4.Exit
30 20 10

1.Push 2.Pop 3.Display 4.Exit
Deleted: 30

1.Push 2.Pop 3.Display 4.Exit
Deleted: 20

1.Push 2.Pop 3.Display 4.Exit
10

1.Push 2.Pop 3.Display 4.Exit
```

RESULT

Thus, PUSH, POP and DISPLAY operations were implemented successfully using an array-based stack.

EXPERIMENT 4: Conversion of Infix to Postfix

AIM

To convert an infix expression into its equivalent postfix expression using a stack.

ALGORITHM

1. Accept the infix expression from the user and initialize an empty stack along with an empty output string.
2. Scan the expression symbol by symbol from left to right.
3. If the scanned symbol is an operand, append it directly to the output.
4. If the symbol is an opening parenthesis, push it onto the stack.
5. If the symbol is a closing parenthesis, pop from the stack and append to the output until an opening parenthesis is encountered, then discard the parenthesis.
6. If the symbol is an operator, repeatedly pop operators from the stack that have higher or equal precedence (respecting right associativity for exponentiation) and append them to the output, then push the current operator onto the stack.
7. After the entire expression has been scanned, pop all the remaining operators from the stack and append them to the output.
8. Display the final string, which is the required postfix expression.

PROGRAM (C)

```
#include <stdio.h>
#include <ctype.h>
#define MAX 100
char st[MAX]; int top = -1;
void push(char c){ st[++top] = c; }
char pop(void){ return st[top--]; }
int prec(char c){
    if (c == '^') return 3;
    if (c == '*' || c == '/') return 2;
    if (c == '+' || c == '-') return 1;
    return 0;
}
int main(void){
    char in[MAX], post[MAX], c;
    int i, j = 0;
    printf("Enter infix expression: ");
    scanf("%99s", in);
    for (i = 0; in[i] != '\0'; i++){
        c = in[i];
        if (isalnum((unsigned char)c)) post[j++] = c;
        else if (c == '(') push(c);
        else if (c == ')'){
            while (top != -1 && st[top] != '(') post[j++] = pop();
            if (top != -1) pop();
        } else {
            while (top != -1 && st[top] != '(' &&
                   (prec(st[top]) > prec(c) ||
                    (prec(st[top]) == prec(c) && c != '^')))
                post[j++] = pop();
            push(c);
        }
    }
    while (top != -1) post[j++] = pop();
    post[j] = '\0';
    printf("Postfix: %s\n", post);
    return 0;
}
```

OUTPUT

```
Enter infix expression: Postfix: AB+C*D-
```

RESULT

Thus, the given infix expression was converted into postfix form using a stack.

EXPERIMENT 5: Postfix Evaluation

AIM

To evaluate a postfix expression using a stack.

ALGORITHM

1. Accept the postfix expression from the user and initialize an empty stack for storing operands.
2. Read the tokens of the expression one by one from left to right.
3. Whenever a token is a number, convert it to an integer and push it onto the stack.
4. Whenever a token is an operator, pop the topmost element as the right operand and the next element as the left operand.
5. Apply the operator on the two operands, checking for division by zero, and push the obtained result back onto the stack.
6. Continue the process until every token of the expression has been processed.
7. When only one value remains in the stack, display it as the final result of the expression.

PROGRAM (C)

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#define MAX 100
int main(void){
    char exp[MAX], *p;
    int st[MAX], top = -1, a, b;
    printf("Enter space-separated postfix expression: ");
    fgets(exp, sizeof exp, stdin);
    for (p = exp; *p;){
        char *end; long value;
        while (isspace((unsigned char)*p)) p++;
        if (*p == '\0') break;
        if (isdigit((unsigned char)*p) ||
            ((*p == '-' || *p == '+') && isdigit((unsigned char)p[1]))){
            value = strtol(p, &end, 10);
            st[++top] = (int)value;
            p = end;
        } else {
            char op = *p++;
            if (top < 1){ printf("Invalid expression\n"); return 1; }
            b = st[top--]; a = st[top--];
            switch (op){
                case '+': st[++top] = a + b; break;
                case '-': st[++top] = a - b; break;
                case '*': st[++top] = a * b; break;
                case '/':
                    if (b == 0){ printf("Division by zero\n"); return 1; }
                    st[++top] = a / b; break;
                default: printf("Invalid operator\n"); return 1;
            }
        }
    }
    if (top == 0) printf("Result = %d\n", st[top]);
    else printf("Invalid expression\n");
    return 0;
}
```

OUTPUT

```
Enter space-separated postfix expression: Result = 11
```

RESULT

Thus, the postfix expression was evaluated successfully using a stack.

EXPERIMENT 6: Simple Queue

AIM

To implement insertion, deletion and display operations in a linear queue using an array.

ALGORITHM

1. Declare an array of fixed size and set both pointers FRONT and REAR to -1 to represent an empty queue.
2. For ENQUEUE, check whether REAR has reached the last position of the array (overflow). If the queue is empty, set FRONT to 0; then increment REAR and insert the new element at that position.
3. For DEQUEUE, check whether the queue is empty (underflow). If not, remove and display the element at FRONT, then increment FRONT.
4. After every deletion, if FRONT moves beyond REAR, reset both pointers back to -1 to mark the queue as empty.
5. For DISPLAY, if the queue is not empty, print the elements starting from FRONT up to REAR in order.
6. Repeat the menu until the user selects the exit option.

PROGRAM (C)

```
#include <stdio.h>
#define MAX 50
int main(void){
    int q[MAX], front = -1, rear = -1, ch, x, i;
    do {
        printf("\n1.Enqueue 2.Dequeue 3.Display 4.Exit\n");
        scanf("%d", &ch);
        if (ch == 1){
            if (rear == MAX-1) printf("Queue overflow\n");
            else {
                printf("Enter value: "); scanf("%d", &x);
                if (front == -1) front = 0;
                q[++rear] = x;
            }
        } else if (ch == 2){
            if (front == -1) printf("Queue underflow\n");
            else {
                printf("Deleted: %d\n", q[front++]);
                if (front > rear) front = rear = -1;
            }
        } else if (ch == 3){
            if (front == -1) printf("Queue is empty\n");
            else { for (i = front; i <= rear; i++) printf("%d ", q[i]); printf("\n"); }
        }
    } while (ch != 4);
    return 0;
}
```

OUTPUT

```
1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter value:
1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter value:
1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter value:
1.Enqueue 2.Dequeue 3.Display 4.Exit
10 20 30
1.Enqueue 2.Dequeue 3.Display 4.Exit
```

```
Deleted: 10

1.Enqueue 2.Dequeue 3.Display 4.Exit
Deleted: 20

1.Enqueue 2.Dequeue 3.Display 4.Exit
30

1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter value:
1.Enqueue 2.Dequeue 3.Display 4.Exit
30 40

1.Enqueue 2.Dequeue 3.Display 4.Exit
```

RESULT

Thus, enqueue, dequeue and display operations were performed successfully in a simple queue.

EXPERIMENT 7: Priority Queue

AIM

To implement a priority queue in which the element with the highest priority (smallest priority number) is deleted first.

ALGORITHM

1. Declare an array in which every entry stores an item together with its priority number, and initialize the count of elements to zero.
2. For insertion, check whether the array has free space. If space is available, read the item and its priority and store them at the end of the array, then increase the count.
3. For deletion, first check whether the queue is empty. If not, scan the array to locate the element having the smallest priority number, which is the highest priority item.
4. Display the located element and remove it from the array by shifting all the following entries one position to the left, then decrease the count.
5. For display, print every item stored in the queue along with its priority number.
6. Repeat the menu until the user chooses to exit.

PROGRAM (C)

```
#include <stdio.h>
#define MAX 50
struct Item { int data, priority; };
int main(void){
    struct Item q[MAX];
    int n = 0, ch, i, best;
    do {
        printf("\n1.Insert 2.Delete 3.Display 4.Exit\n");
        scanf("%d", &ch);
        if (ch == 1){
            if (n == MAX) printf("Queue is full\n");
            else {
                printf("Enter data and priority: ");
                scanf("%d%d", &q[n].data, &q[n].priority);
                n++;
            }
        } else if (ch == 2){
            if (n == 0) printf("Queue is empty\n");
            else {
                best = 0;
                for (i = 1; i < n; i++)
                    if (q[i].priority < q[best].priority) best = i;
                printf("Deleted: %d\n", q[best].data);
                for (i = best; i < n-1; i++) q[i] = q[i+1];
                n--;
            }
        } else if (ch == 3){
            if (n == 0) printf("Queue is empty\n");
            for (i = 0; i < n; i++)
                printf("Data=%d Priority=%d\n", q[i].data, q[i].priority);
        }
    } while (ch != 4);
    return 0;
}
```

OUTPUT

```
1.Insert 2.Delete 3.Display 4.Exit
Enter data and priority:
1.Insert 2.Delete 3.Display 4.Exit
```

```
Enter data and priority:
1.Insert 2.Delete 3.Display 4.Exit
Enter data and priority:
1.Insert 2.Delete 3.Display 4.Exit
Data=10 Priority=2
Data=20 Priority=1
Data=30 Priority=3

1.Insert 2.Delete 3.Display 4.Exit
Deleted: 20

1.Insert 2.Delete 3.Display 4.Exit
Data=10 Priority=2
Data=30 Priority=3

1.Insert 2.Delete 3.Display 4.Exit
```

RESULT

Thus, insertion, priority-based deletion and display operations were implemented successfully.

EXPERIMENT 8: Singly Linked List

AIM

To create a singly linked list and perform insertion at the end, deletion by value and traversal.

ALGORITHM

1. Define a node structure containing a data field and a pointer to the next node, and initialize the head pointer to NULL.
2. For insertion at the end, allocate memory for a new node, store the value in it and set its next pointer to NULL. If the list is empty, make the new node the head; otherwise traverse to the last node and link the new node after it.
3. For deletion by value, traverse the list while remembering the previous node, until the node holding the value is found.
4. If the value is found, adjust the pointers so that the previous node points to the successor of the located node (or update the head if it is the first node), release the memory of the removed node and confirm the deletion; otherwise display a not-found message.
5. For traversal, start from the head and print every node following the next pointers until NULL is reached.
6. Repeat the menu until exit, and release all remaining nodes before terminating.

PROGRAM (C)

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *next; };
int main(void){
    struct Node *head = NULL, *tail, *p, *prev, *node;
    int ch, x;
    do {
        printf("\n1.Insert end 2.Delete value 3.Display 4.Exit\n");
        scanf("%d", &ch);
        if (ch == 1){
            printf("Enter value: "); scanf("%d", &x);
            node = malloc(sizeof *node);
            if (!node) return 1;
            node->data = x; node->next = NULL;
            if (!head) head = node;
            else { for (tail = head; tail->next; tail = tail->next){} tail->next = node; }
        } else if (ch == 2){
            printf("Enter value: "); scanf("%d", &x);
            p = head; prev = NULL;
            while (p && p->data != x){ prev = p; p = p->next; }
            if (!p) printf("Value not found\n");
            else {
                if (!prev) head = p->next; else prev->next = p->next;
                free(p); printf("Deleted\n");
            }
        } else if (ch == 3){
            for (p = head; p; p = p->next) printf("%d -> ", p->data);
            printf("NULL\n");
        }
    } while (ch != 4);
    while (head){ p = head; head = head->next; free(p); }
    return 0;
}
```

OUTPUT

```
1.Insert end 2.Delete value 3.Display 4.Exit
```

```
Enter value:
1.Insert end 2.Delete value 3.Display 4.Exit
Enter value:
1.Insert end 2.Delete value 3.Display 4.Exit
Enter value:
1.Insert end 2.Delete value 3.Display 4.Exit
10 -> 20 -> 30 -> NULL

1.Insert end 2.Delete value 3.Display 4.Exit
Enter value: Deleted

1.Insert end 2.Delete value 3.Display 4.Exit
10 -> 30 -> NULL

1.Insert end 2.Delete value 3.Display 4.Exit
Enter value: Value not found

1.Insert end 2.Delete value 3.Display 4.Exit
```

RESULT

Thus, insertion, deletion and traversal were implemented successfully using a singly linked list.

EXPERIMENT 9: Doubly Linked List

AIM

To create a doubly linked list and perform insertion at the end, deletion by value and forward traversal.

ALGORITHM

1. Define a node structure with a data field, a pointer to the previous node and a pointer to the next node. Initialize both HEAD and TAIL to NULL.
2. For insertion at the end, allocate a new node, store the value and set its next pointer to NULL and its previous pointer to the current TAIL. If the list is empty, make the new node the HEAD; otherwise link it after TAIL. Finally update TAIL to the new node.
3. For deletion by value, traverse the list from HEAD following next pointers until the node containing the value is located.
4. If found, bypass the node by connecting its previous node directly to its next node. Adjust HEAD if the first node is removed and TAIL if the last node is removed, then free the node memory; otherwise report that the value is not present.
5. For traversal, begin at HEAD, print each node's data and move forward using the next pointers until the end of the list.
6. Repeat the menu until exit and free every remaining node before the program ends.

PROGRAM (C)

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *prev, *next; };
int main(void){
    struct Node *head = NULL, *tail = NULL, *p, *node;
    int ch, x;
    do {
        printf("\n1.Insert end 2.Delete value 3.Display 4.Exit\n");
        scanf("%d", &ch);
        if (ch == 1){
            printf("Enter value: "); scanf("%d", &x);
            node = malloc(sizeof *node);
            if (!node) return 1;
            node->data = x; node->prev = tail; node->next = NULL;
            if (tail) tail->next = node; else head = node;
            tail = node;
        } else if (ch == 2){
            printf("Enter value: "); scanf("%d", &x);
            for (p = head; p && p->data != x; p = p->next){}
            if (!p) printf("Value not found\n");
            else {
                if (p->prev) p->prev->next = p->next; else head = p->next;
                if (p->next) p->next->prev = p->prev; else tail = p->prev;
                free(p); printf("Deleted\n");
            }
        } else if (ch == 3){
            for (p = head; p; p = p->next) printf("%d <-> ", p->data);
            printf("NULL\n");
        }
    } while (ch != 4);
    while (head){ p = head; head = head->next; free(p); }
    return 0;
}
```

OUTPUT

```
1.Insert end 2.Delete value 3.Display 4.Exit
Enter value:
1.Insert end 2.Delete value 3.Display 4.Exit
Enter value:
1.Insert end 2.Delete value 3.Display 4.Exit
Enter value:
1.Insert end 2.Delete value 3.Display 4.Exit
10 <-> 20 <-> 30 <-> NULL

1.Insert end 2.Delete value 3.Display 4.Exit
Enter value: Deleted

1.Insert end 2.Delete value 3.Display 4.Exit
10 <-> 30 <-> NULL

1.Insert end 2.Delete value 3.Display 4.Exit
```

RESULT

Thus, insertion, deletion and traversal were implemented successfully using a doubly linked list.

EXPERIMENT 10: Circular Queue

AIM

To implement enqueue, dequeue and display operations using an array-based circular queue.

ALGORITHM

1. Declare an array of fixed size and initialize FRONT and REAR to -1, representing an empty circular queue.
2. The queue is full when the position (REAR + 1) modulo MAX coincides with FRONT.
3. For ENQUEUE, check the overflow condition. If the queue is empty, set FRONT to 0. Update REAR using (REAR + 1) modulo MAX and insert the new element at this position.
4. For DEQUEUE, check the underflow condition. Remove and display the element at FRONT. If it was the only element, reset both pointers to -1; otherwise advance FRONT using (FRONT + 1) modulo MAX.
5. For DISPLAY, start at FRONT and print elements one by one, wrapping around to the beginning of the array using modulo arithmetic, until REAR is reached.
6. Repeat the menu until the user selects exit.

PROGRAM (C)

```
#include <stdio.h>
#define MAX 5
int main(void){
    int q[MAX], front = -1, rear = -1, ch, x, i;
    do {
        printf("\n1.Enqueue 2.Dequeue 3.Display 4.Exit\n");
        scanf("%d", &ch);
        if (ch == 1){
            if ((rear+1) % MAX == front) printf("Queue is full\n");
            else {
                printf("Enter value: "); scanf("%d", &x);
                if (front == -1) front = 0;
                rear = (rear+1) % MAX; q[rear] = x;
            }
        } else if (ch == 2){
            if (front == -1) printf("Queue is empty\n");
            else {
                printf("Deleted: %d\n", q[front]);
                if (front == rear) front = rear = -1;
                else front = (front+1) % MAX;
            }
        } else if (ch == 3){
            if (front == -1) printf("Queue is empty\n");
            else {
                i = front;
                while (1){
                    printf("%d ", q[i]);
                    if (i == rear) break;
                    i = (i+1) % MAX;
                }
                printf("\n");
            }
        }
    } while (ch != 4);
    return 0;
}
```

OUTPUT

```
1.Enqueue 2.Dequeue 3.Display 4.Exit
```

```
Enter value:
1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter value:
1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter value:
1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter value:
1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter value:
1.Enqueue 2.Dequeue 3.Display 4.Exit
10 20 30 40 50

1.Enqueue 2.Dequeue 3.Display 4.Exit
Deleted: 10

1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter value:
1.Enqueue 2.Dequeue 3.Display 4.Exit
20 30 40 50 60

1.Enqueue 2.Dequeue 3.Display 4.Exit
```

RESULT

Thus, circular queue operations were implemented successfully using modulo arithmetic.

EXPERIMENT 11: Tree Traversal

AIM

To create a binary search tree and display its elements using inorder, preorder and postorder traversals.

ALGORITHM

1. Define a tree node structure containing a data field and pointers to the left and right child nodes.
2. Insert each value into the binary search tree: if the value is smaller than the current node's data, move to the left subtree; if it is larger, move to the right subtree; repeat recursively until an empty position is found and create the new node there.
3. For INORDER traversal, first visit the left subtree recursively, then display the root node, and finally visit the right subtree recursively.
4. For PREORDER traversal, first display the root node, then visit the left subtree recursively, and finally visit the right subtree recursively.
5. For POSTORDER traversal, first visit the left subtree recursively, then the right subtree recursively, and display the root node at the end.
6. Read the values from the user, build the tree, display all three traversals and release the allocated memory.

PROGRAM (C)

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node *left, *right; };
struct Node *insert(struct Node *root, int x){
    if (!root){
        root = malloc(sizeof *root);
        if (!root) exit(1);
        root->data = x; root->left = root->right = NULL;
    } else if (x < root->data) root->left = insert(root->left, x);
    else if (x > root->data) root->right = insert(root->right, x);
    return root;
}
void inorder(struct Node *r){
    if (r){ inorder(r->left); printf("%d ", r->data); inorder(r->right); }
}
void preorder(struct Node *r){
    if (r){ printf("%d ", r->data); preorder(r->left); preorder(r->right); }
}
void postorder(struct Node *r){
    if (r){ postorder(r->left); postorder(r->right); printf("%d ", r->data); }
}
void freeTree(struct Node *r){
    if (r){ freeTree(r->left); freeTree(r->right); free(r); }
}
int main(void){
    struct Node *root = NULL;
    int n, x, i;
    printf("Enter number of nodes: "); scanf("%d", &n);
    printf("Enter values: ");
    for (i = 0; i < n; i++){ scanf("%d", &x); root = insert(root, x); }
    printf("Inorder: "); inorder(root);
    printf("\nPreorder: "); preorder(root);
    printf("\nPostorder: "); postorder(root);
    printf("\n");
    freeTree(root);
    return 0;
}
```

OUTPUT

```
Enter number of nodes: Enter values: Inorder: 2 3 4 5 7
Preorder: 5 3 2 4 7
Postorder: 2 4 3 7 5
```

RESULT

Thus, inorder, preorder and postorder traversals of a binary search tree were implemented successfully.

EXPERIMENT 12: Graph Search Algorithms (DFS and BFS)

AIM

To represent an undirected graph using an adjacency matrix and traverse it using DFS and BFS.

ALGORITHM

1. Read the number of vertices of the graph and then read the adjacency matrix, where a value of 1 indicates an edge between the corresponding vertices.
2. For Depth First Search, mark the starting vertex as visited, display it, and recursively apply the same procedure to every adjacent vertex that has not yet been visited.
3. For Breadth First Search, first clear the visited marks and initialize an empty queue. Mark the starting vertex as visited and enqueue it.
4. While the queue is not empty, dequeue a vertex, display it, and enqueue every adjacent vertex that has not been visited, marking each one as visited at the moment it enters the queue.
5. Execute both DFS and BFS beginning from the starting vertex chosen by the user and display the two traversal orders.

PROGRAM (C)

```
#include <stdio.h>
#define MAX 20
int graph[MAX][MAX], visited[MAX], n;
void dfs(int v){
    int i;
    visited[v] = 1;
    printf("%d ", v);
    for (i = 0; i < n; i++)
        if (graph[v][i] && !visited[i]) dfs(i);
}
void bfs(int start){
    int q[MAX], front = 0, rear = 0, i, v;
    for (i = 0; i < n; i++) visited[i] = 0;
    visited[start] = 1; q[rear++] = start;
    while (front < rear){
        v = q[front++]; printf("%d ", v);
        for (i = 0; i < n; i++)
            if (graph[v][i] && !visited[i]){
                visited[i] = 1; q[rear++] = i;
            }
    }
}
int main(void){
    int i, j, start;
    printf("Enter number of vertices: "); scanf("%d", &n);
    printf("Enter adjacency matrix:\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++) scanf("%d", &graph[i][j]);
    printf("Enter starting vertex (0 to %d): ", n-1); scanf("%d", &start);
    for (i = 0; i < n; i++) visited[i] = 0;
    printf("DFS: "); dfs(start);
    printf("\nBFS: "); bfs(start);
    printf("\n");
    return 0;
}
```

OUTPUT

```
Enter number of vertices: Enter adjacency matrix:
Enter starting vertex (0 to 4): DFS: 0 1 3 4 2
BFS: 0 1 2 3 4
```

RESULT

Thus, graph traversal was performed successfully using Depth First Search and Breadth First Search.

EXPERIMENT 13: Bubble, Selection and Heap Sort

AIM

To sort a list of integers in ascending order using bubble sort, selection sort and heap sort.

ALGORITHM

1. Read the count of elements and the elements themselves into an array, and keep an unchanged copy of the original list.
2. BUBBLE SORT: perform passes over the array, comparing each pair of adjacent elements and exchanging them whenever they appear in the wrong order, so that the largest element settles at its correct place after every pass.
3. SELECTION SORT: repeatedly locate the smallest element in the unsorted portion of the array and exchange it with the first unsorted element, extending the sorted portion by one in each iteration.
4. HEAP SORT: first rearrange the array into a max heap. Then repeatedly exchange the root (largest value) with the last element of the unsorted region and restore the heap property on the reduced region.
5. Apply each sorting method to a fresh copy of the original array and display the sorted array produced by every method.

PROGRAM (C)

```
#include <stdio.h>
#define MAX 100
void print(int a[], int n){
    int i;
    for (i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\n");
}
void copy(int src[], int dst[], int n){
    int i; for (i = 0; i < n; i++) dst[i] = src[i];
}
void bubble(int a[], int n){
    int i, j, t;
    for (i = 0; i < n-1; i++)
        for (j = 0; j < n-i-1; j++)
            if (a[j] > a[j+1]){ t = a[j]; a[j] = a[j+1]; a[j+1] = t; }
}
void selection(int a[], int n){
    int i, j, min, t;
    for (i = 0; i < n-1; i++){
        min = i;
        for (j = i+1; j < n; j++)
            if (a[j] < a[min]) min = j;
        t = a[i]; a[i] = a[min]; a[min] = t;
    }
}
void heapify(int a[], int n, int i){
    int largest = i, l = 2*i+1, r = 2*i+2, t;
    if (l < n && a[l] > a[largest]) largest = l;
    if (r < n && a[r] > a[largest]) largest = r;
    if (largest != i){
        t = a[i]; a[i] = a[largest]; a[largest] = t;
        heapify(a, n, largest);
    }
}
void heapSort(int a[], int n){
    int i, t;
    for (i = n/2-1; i >= 0; i--) heapify(a, n, i);
```

```
    for (i = n-1; i > 0; i--){
        t = a[0]; a[0] = a[i]; a[i] = t;
        heapify(a, i, 0);
    }
}
int main(void){
    int original[MAX], a[MAX], b[MAX], c[MAX], n, i;
    printf("Enter number of elements: "); scanf("%d", &n);
    printf("Enter elements: ");
    for (i = 0; i < n; i++) scanf("%d", &original[i]);
    copy(original, a, n); bubble(a, n);
    copy(original, b, n); selection(b, n);
    copy(original, c, n); heapSort(c, n);
    printf("Bubble sort: "); print(a, n);
    printf("Selection sort: "); print(b, n);
    printf("Heap sort: "); print(c, n);
    return 0;
}
```

OUTPUT

```
Enter number of elements: Enter elements: Bubble sort: 1 2 3 4 5
Selection sort: 1 2 3 4 5
Heap sort: 1 2 3 4 5
```

RESULT

Thus, bubble sort, selection sort and heap sort were implemented and all produced the elements in ascending order.